



[DATUM]

Projektschlussbericht

306 KLEINPROJEKTE IM EIGENEN BERUFSUMFELD ABWICKELN

JENI

Inhaltsverzeichnis

Inhalt

Inhaltsverzeichnis	1
1 Projektübersicht	1
1.1 Ausgangslage	1
1.2 Projektziele	2
1.3 1.3 Kurzvorstellung des Projekts.....	3
2 Planung.....	3
2.1 Projektorganisation.....	3
2.2 Projektplanung	4
2.3 Risiken und Herausforderungen.....	4
3 Umsetzung	5
3.1 Durchgeführte Arbeiten.....	5
3.2 Verwendete Technologien und Werkzeuge.....	5
3.3 Herausforderungen und Lösungen	6
3.4 Abweichungen von der Planung	6
4 Ergebnisse	7
4.1 Erreichte Ziele.....	7
4.2 Produkt / Lösung	7
4.3 Tests und Qualitätssicherung	8
5 Reflexion (individuell)	9
Reflexion von Boris (Projektleiter).....	9
6 Schlusswort	9
Anhang.....	10

1 Projektübersicht

1.1 Ausgangslage

Die Koordination von Gruppenaktivitäten und die dazugehörige Terminfindung stellen in Freundeskreisen und Teams eine alltägliche Herausforderung dar. Die

Abprache über klassische Instant-Messenger-Dienste wie WhatsApp, Telegram oder Threema führt in der Praxis oft zu unübersichtlichen und chaotischen Chat-Verläufen. Wichtige Details zu geplanten Aktivitäten, alternative Zeitfenster und die Zu- oder Absagen von einzelnen Teilnehmern gehen in der Masse der alltäglichen Nachrichten schnell unter. Bestehende Umfrage-Tools in diesen Messengern sind funktional stark eingeschränkt, bieten keine langfristige Übersicht und lassen sich nicht an feste Themengruppen binden.

Spezialisierte Plattformen wie Doodle lösen zwar das Problem der reinen Terminabstimmung, weisen jedoch erhebliche Schwächen im Kontext von wiederkehrenden Gruppen auf. Sie sind oft mit störender Werbung überladen, wirken unpersönlich und bieten keine Möglichkeit, Gruppen dauerhaft zu verwalten, Interessen-Tags zu pflegen oder Aktivitätsvorschläge zentral zu sammeln. Dies führt dazu, dass für jede neue Aktivität eine neue Umfrage gestartet und der Link mühsam verteilt werden muss. Ein zentraler digitaler Workspace für Freundesgruppen fehlte bislang im persönlichen Umfeld.

Aus dieser realen Frustration heraus entstand die Projektidee für MeetPoint. Ziel des Projekts war es, eine webbasierte Single-Page-Application (SPA) zu entwickeln, die den gesamten Prozess der Freizeitplanung strukturiert und vereinfacht: von der Erstellung einer Themengruppe über die Definition gemeinsamer Interessen bis hin zur automatischen Ermittlung des optimalen Termins für eine Aktivität. Die Umsetzung erfolgte als Kleinprojekt im Team während des Moduls 306.

1.2 Projektziele

Das primäre Ziel des Projekts war die erfolgreiche Entwicklung und lokale Bereitstellung der Webapplikation MeetPoint bis zum Ende des Moduls 306. Die Kriterien für den Projekterfolg wurden anhand der SMART-Methode wie folgt definiert:

Spezifisch (Specific): Entwicklung einer funktionsfähigen Web-App, die es Benutzern erlaubt, Konten zu erstellen, sich anzumelden, Themengruppen zu gründen, Interessen-Tags zu erfassen, konkrete Aktivitäten vorzuschlagen und über mehrere Terminvorschläge reaktiv abzustimmen.

Messbar (Measurable): Die Funktionalität wird durch 23 definierte funktionale Anforderungen (F1–F24) und 12 nicht-funktionale Anforderungen (NF1–NF12) gemessen. Der Projekterfolg ist erreicht, wenn alle als 'Muss' deklarierten Anforderungen (Priorität Hoch) implementiert und durch nachvollziehbare Testfälle erfolgreich verifiziert wurden.

Attraktiv (Attractive): Durch den Einsatz eines modernen Dark-Themes mit transparenten 'Glassmorphism'-Effekten und reaktiven UI-Elementen soll die

Applikation eine hohe Usability und Ästhetik bieten, die zur regelmässigen Nutzung motiviert.

Realistisch (Realistic): Die technische Umsetzung erfolgt mit einem fokussierten Tech-Stack (Vue.js 3 im Frontend, PostgreSQL und PostgREST im Backend) über eine Docker-Compose-Infrastruktur, was eine Realisierung innerhalb des vorgegebenen Zeitrahmens von sechs Wochen ermöglicht.

Terminiert (Time-bound): Der Meilensteinplan sieht eine schrittweise Abwicklung vor, die mit dem Abschluss-Review und der finalen Abgabe des Projektschlussberichts am Ende der Modullaufzeit abschliesst.

1.3 Kurzvorstellung des Projekts

MeetPoint ist ein innovatives Planungswerkzeug für Gruppen. Die Applikation trennt strikt zwischen der Benutzeroberfläche (Frontend) und der Datenhaltung (Backend). Das Frontend wurde als reaktive SPA mit Vue 3 und TypeScript entwickelt. Auf ein schwerfälliges CSS-Framework wurde verzichtet; stattdessen wurde ein massgeschneidertes Styling-System mit Vanilla CSS implementiert, das einen durchgehenden, eleganten Dark-Mode mit Glassmorphism (semi-transparente Panels mit Milchglas-Effekt) realisiert.

Im Backend setzen wir auf eine PostgreSQL-Datenbank, die in einem Docker-Container läuft. Anstatt eine eigene API in Node.js oder Python zu programmieren, wird das Open-Source-Werkzeug PostgREST verwendet. PostgREST spiegelt das PostgreSQL-Datenbankschema direkt als RESTful API auf HTTP-Basis. Zur Absicherung der Datenintegrität wurden manipulationssichere Stored Procedures (RPCs) in PL/pgSQL geschrieben. Das Gesamtsystem kann über eine einzige docker compose up-Anweisung lokal gestartet werden.

2 Planung

2.1 Projektorganisation

Das Projektteam setzte sich aus vier Mitgliedern der Klasse BM2SD24a zusammen. Um eine effiziente Abwicklung zu garantieren, wurden feste Rollen und Verantwortlichkeiten definiert:

Boris – Projektleiter: Verantwortlich für die Gesamtkoordination, die Einhaltung der Meilensteine, die Arbeitsverteilung, die Git-Repository-Pflege und die Erstellung der Präsentationsmedien.

Fabio – Stellvertretender Projektleiter & Analyst: Verantwortlich für die Anforderungsanalyse, die Erstellung des Bewertungsschemas für die Nutzwertanalyse, die Risikoidentifikation und die Unterstützung der Projektleitung.

Dmytro – Technischer Leiter & Database Engineer: Verantwortlich für das relationale Datenbankdesign, die PostgREST-Konfiguration, das Docker-Setup und die Bereitstellung der API-Endpunkte.

Noël – Qualitätssicherung & Testleiter: Verantwortlich für die Testfall-Spezifikation, das Führen des Testprotokolls, die Verifikation der Anforderungen und die Mitwirkung an der Projektdokumentation.

Die Zusammenarbeit erfolgte primär während der Unterrichtsstunden sowie über Microsoft Teams für den Dokumentenaustausch und Git/GitHub für die Codeverwaltung. Wöchentliche Status-Meetings stellten sicher, dass eventuelle Verzögerungen frühzeitig erkannt wurden.

2.2 Projektplanung

Die zeitliche Abwicklung orientierte sich an den sechs Meilensteinen des Moduls 306. Die detaillierte Aufgabenplanung wurde in Form von 23 Arbeitspaketen (AP 1.1 bis AP 6.7) in einem Excel-Projektplan ausgearbeitet und visualisiert (siehe Gantt-Diagramm im Anhang). Die Hauptphasen umfassten:

1. Initialisierung & Analyse: Ausarbeitung des Projektauftrags, Durchführung der Stakeholder- und Nutzwertanalyse (Vergleich von WebStorm + DBeaver mit anderen Editoren) sowie Definition der funktionalen und nicht-funktionalen Anforderungen.
2. Konzeption & DB-Design: Erstellung des Entity-Relationship-Diagramms (ERD) und Festlegung der Tabellenstrukturen und Trigger für die PostgreSQL-Datenbank.
3. Implementierung: Parallele Entwicklung des Frontends (Vue-Komponenten) und des Backends (SQL-Tabellen, PostgREST-API und RPC-Sicherheitsfunktionen).
4. Testphase: Systematische E2E-Tests der Muss-Anforderungen anhand des Testprotokolls und anschliessende Behebung gefundener Fehler.
5. Abschluss & Dokumentation: Erstellung der Abschlusspräsentation und des Projektschlussberichts.

2.3 Risiken und Herausforderungen

Im Rahmen der Risikoanalyse wurden vier Hauptrisiken identifiziert und bewertet: Zeitverzögerung (Eintrittswahrscheinlichkeit Hoch), Technische Probleme mit PostgREST (Mittel), Kommunikationsprobleme im Team (Gering) und Fehler im Datenbankdesign (Mittel).

Als wichtigste Präventivmassnahme wurde ein Puffer im Terminplan definiert. Zudem wurde vereinbart, technische Prototypen (z.B. die PostgREST-Anbindung) bereits in der ersten Woche zu testen. Für die Datenkonsistenz wurden Fremdschlüsselbedingungen und Kaskadierungen (ON DELETE CASCADE) von Beginn an strikt im SQL-Schema verankert. Die vollständigen Risiko- und Stakeholderanalysen sind im Anhang beigefügt.

3 Umsetzung

3.1 Durchgeführte Arbeiten

Die Umsetzung des Projekts erfolgte in einer logischen, aufeinander aufbauenden Reihenfolge:

Zunächst implementierte Dmytro das relationale Datenbankschema (001_create_tables.sql). Es wurden Tabellen für users, groups, group_members, interests, activities, activity_interests, appointments und availabilities erstellt. Um die API-Endpunkte sauber abzugrenzen, wurde ein separates Datenbankschema api definiert, das als Views auf das interne Schema app verweist.

Anschliessend wurde die lokale Laufzeitumgebung mittels Docker Compose konfiguriert. Hierbei traten erste Probleme mit der Initialisierungsreihenfolge auf, da PostgREST startete, bevor die PostgreSQL-Datenbank bereit war. Dies wurde durch den Einbau eines Docker-Healthchecks für die Datenbank gelöst.

Im Anschluss entwickelte das Team das Vue 3 Frontend. Es wurden reaktive Komponenten für das User-Login, die Gruppenverwaltung und die Terminabstimmung erstellt. Dabei kam das reaktive State-Management von Vue (mittels ref und computed) zum Einsatz, um Änderungen an den Abstimmungsergebnissen ohne Seiten-Reload in Echtzeit darzustellen.

Im letzten Schritt der Implementierung wurden die Berechtigungsprüfungen für das Löschen von Gruppen, Interessen und Terminen umgesetzt. Da im MVP-Status kein JWT-Token verwendet wird, schrieben wir sichere PostgreSQL-RPCs (Stored Procedures), die serverseitig prüfen, ob der anfragende Nutzer die Berechtigung zum Löschen hat.

3.2 Verwendete Technologien und Werkzeuge

Der gewählte Technologie-Stack erwies sich als äusserst effizient. Die Nutzwertanalyse bestätigte die Wahl von WebStorm als IDE und DBeaver als Datenbank-Client aufgrund der hervorragenden Unterstützung reaktiver Webtechnologien und SQL-Datenbankschemata.

Vue 3 (Vite, TypeScript): Ermöglichte eine schnelle, komponentenbasierte Entwicklung des Frontends. Durch Vite profitierten wir von extrem schnellen Hot-Module-Replacement-Ladezeiten.

PostgreSQL & PostgREST: PostgREST generierte die API-Endpunkte direkt aus dem SQL-Schema. Dies sparte uns das manuelle Programmieren von Backend-Routen und Controllern (z.B. in Express.js) und reduzierte die Fehleranfälligkeit.

Docker Desktop: Erlaubte die einfache Virtualisierung von Datenbank und PostgREST. Dadurch ist die Applikation plattformunabhängig und kann auf jedem System mit minimalem Aufwand gestartet werden.

3.3 Herausforderungen und Lösungen

Während der Umsetzung stiessen wir auf wesentliche technische und organisatorische Herausforderungen:

1. Sicherheit und Berechtigungsprüfung im MVP: PostgREST spiegelt standardmässig alle Tabellenoperationen direkt. Ohne ein JWT-Token-System konnte jeder Benutzer über die Schnittstelle fremde Daten löschen. Lösung: Wir haben der PostgREST-Rolle `meetpoint_web` die direkten DELETE-Privilegien entzogen. Stattdessen wurden PL/pgSQL-Funktionen (RPCs) in PostgreSQL implementiert, die bei jedem Aufruf die Benutzerberechtigung prüfen und erst bei Erfolg den Löschbefehl intern ausführen.
2. Fehlende Gruppen bei Nicht-Besitzern: Nach dem Hinzufügen eines Mitglieds zu einer Gruppe konnte dieses die Gruppe in seinem Dashboard nicht sehen. Lösung: Der Fehler lag in der API-Abfrage, die Gruppen nur nach `owner_id` filterte. Wir haben `fetchGroups` in `groupApi.ts` so umgeschrieben, dass zuerst alle Einträge des Benutzers aus der Tabelle `group_members` geladen und anschliessend alle zugehörigen Gruppen via SQL-Operator `in(...)` in einem Rutsch abgefragt werden.
3. Datenbank-Setup-Fehler (Dependency-Reihenfolge): Beim Zurücksetzen der Datenbank stürzte die Container-Initialisierung ab. Lösung: Das Berechtigungsskript (003) lief alphabetisch vor den Löschfunktionen (006), wodurch Privilegien für nicht existierende Funktionen vergeben werden sollten. Wir haben das Skript in `008_postgrest_permissions.sql` umbenannt und die Rollendefinition an den Anfang von `001_create_tables.sql` verschoben, um eine saubere Reihenfolge sicherzustellen.
4. Organisatorische Herausforderungen (Team-Abwesenheiten): Da im Verlauf des Projekts mehrere Teammitglieder verletzungs- oder krankheitsbedingt ausfielen, konnte das geplante Vorgehen – bei dem der Projektleiter (Boris) Prompts für ChatGPT/Gemini schrieb, diese an die Mitglieder verteilte und diese den Code implementierten – nicht aufrechterhalten werden. Lösung: Boris übernahm die Aufgaben der abwesenden Mitglieder und implementierte die verbleibenden Komponenten (z.B. die reaktive Abstimmung und die Modal-Popups) selbst, um den Zeitplan einzuhalten.

3.4 Abweichungen von der Planung

Aufgrund der unerwarteten Abwesenheiten im Team kam es zu einer Verschiebung der Arbeitslast auf die Projektleitung. Während Dmytro das solide Datenbank-Fundament und Fabio die Risikoanalyse lieferten, mussten die Frontend-Integration und die Fehlerbehebungen in den letzten Phasen grösstenteils in Personalunion von Boris durchgeführt werden. Trotz dieser Abweichungen konnten alle Meilensteine pünktlich erreicht werden.

4 Ergebnisse

4.1 Erreichte Ziele

Alle 17 funktionalen Muss-Anforderungen (F1–F4, F6–F8, F10, F12, F13, F15–F17, F19, F20, F23) wurden vollständig und fehlerfrei umgesetzt. Zudem konnten wichtige Soll-Anforderungen (wie F21/F22 – das sichere Löschen von eigenen Einträgen über Modals statt unschöner Browser-Alerts) erfolgreich integriert werden. Die Applikation läuft stabil und fehlerfrei.

4.2 Produkt / Lösung

Das fertige Produkt MeetPoint bietet eine intuitive Benutzeroberfläche. Nach dem Login sieht der Benutzer seine Gruppen und die Mitgliederliste. Wählt er eine Gruppe aus, laden sich die zugeordneten Aktivitäten. Innerhalb einer Aktivität können alternative Termine eingetragen werden. Jedes Mitglied kann mit Daumen-Symbolen abstimmen, woraufhin die App in Echtzeit die Stimmen aggregiert, die Namen der Abstimmenden anzeigt und den besten Termin mit einem Pokal-Symbol (🏆 Bester Termin) hervorhebt.

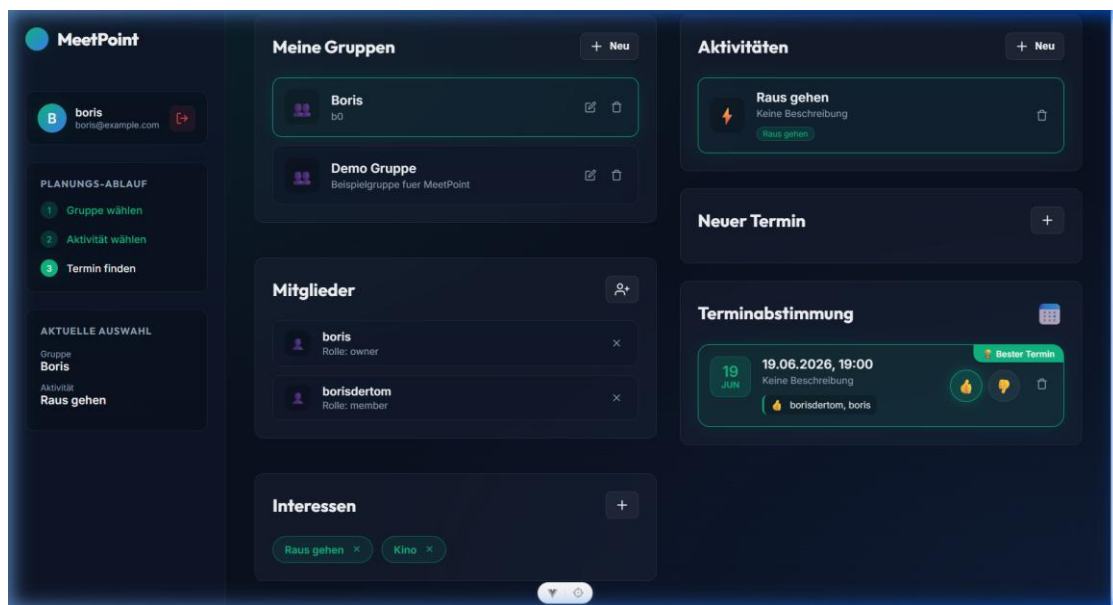


Abbildung 3: Detailübersicht einer Gruppe mit Aktivitäten, Interessen-Chips, Terminvorschlägen, Live-Stimmverteilung und dem berechneten Gewinner-Termin (Bester Termin).



Abbildung 2: Das MeetPoint-Dashboard nach erfolgreichem Login. Es zeigt die Gruppen des Benutzers, die Mitgliederliste der gewählten Gruppe sowie den aktuellen Planungsablauf.

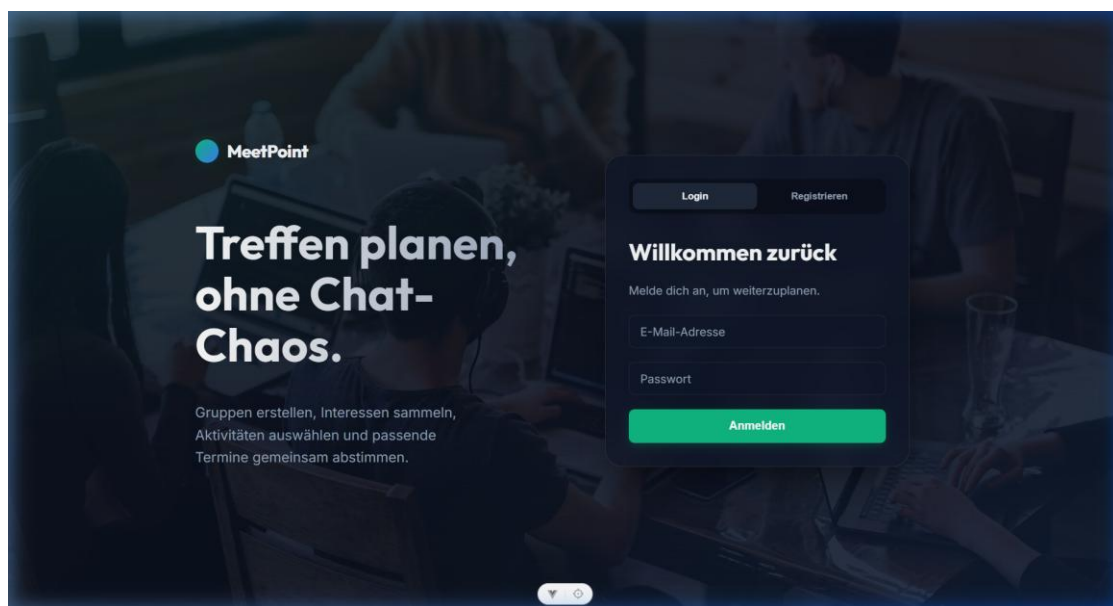


Abbildung 1: Login- und Registrierungsbereich der MeetPoint Webapplikation mit einem eleganten Glassmorphism-Design im Dark-Mode.

4.3 Tests und Qualitätssicherung

Zur Qualitätssicherung wurde ein systematisches Testprotokoll geführt. Noël testete alle Muss-Anforderungen manuell im Browser. Jede Funktion (z.B. das Verhalten bei fehlerhafter Passworteingabe, die Validierung von E-Mail-Adressen und die Zugriffssperre für fremde Gruppen) wurde verifiziert. Alle Testfälle endeten mit dem Status 'Erfolgreich'. Der Code wurde zudem auf Typsicherheit und Struktur geprüft.

5 Reflexion (individuell)

Reflexion von Boris (Projektleiter)

Mein Beitrag zum Projekt

Mein persönlicher Beitrag zum Projekt umfasste die Gesamtkoordination, die Meilensteinplanung und die kontinuierliche Repo-Pflege auf GitHub. Da ich im Team die Rolle des Prompt-Engineers übernahm, schrieb ich präzise Prompts für generative KI-Systeme (ChatGPT/Gemini) und verteilte die erhaltenen Code-Lösungen an meine Teammitglieder für deren Arbeitspakete. In der zweiten Projekthälfte fielen verletzungs- und krankheitsbedingt mehrere Teammitglieder aus, weshalb ich die Implementierung des reaktiven Frontends, der Modal-Popups und der API-Integrationen grösstenteils selbst übernehmen musste, um den Abgabetermin einzuhalten.

Was ist gut gelungen?

Gut gelungen ist mir die schnelle Anpassung an diese unvorhergesehene Sondersituation. Die Organisation und Strukturierung des Projektes im Vorfeld erwies sich als Rettungsanker, da ich die Aufgaben der abwesenden Kollegen sofort nahtlos übernehmen und fertigstellen konnte. Das Endergebnis läuft stabil und entspricht vollkommen unseren Designansprüchen.

Was würde ich beim nächsten Mal anders machen?

Beim nächsten Mal würde ich die Aufgabenpakete noch kleinteiliger schneiden und von Anfang an eine gegenseitige Stellvertretung für Code-Aufgaben einrichten. Auch wenn das Prompten der KI sehr effizient war, erfordert die Integration von generiertem Code in ein Vue 3 Projekt fundierte Eigeninitiative, wenn unerwartete Ausfälle auftreten.

Wichtigste Erkenntnisse

Wichtigste Erkenntnisse: Eine detaillierte und sorgfältige Planung im Vorfeld (Meilensteine, Anforderungen, DB-Design) zahlt sich in der Umsetzungsphase doppelt aus, da sie bei unvorhersehbaren Problemen und Ausfällen als klarer Kompass dient.

6 Schlusswort

Das Projekt MeetPoint war ein voller Erfolg. Trotz der personellen Herausforderungen und Ausfälle konnten wir ein voll funktionsfähiges, sicheres und optisch ansprechendes MVP abliefern. Der Einsatz moderner Technologien wie Vue 3, PostgREST und Docker hat sich als goldrichtig erwiesen. Wir bedanken uns herzlich

bei unserer Lehrperson für die Begleitung und freuen uns über ein gelungenes Softwareprojekt.

Anhang

Relevante Projektdokumente sowie technische Nachweise sind diesem Bericht beigelegt oder im Repository hinterlegt:

- Projektauftrag
- Anforderungen
- Stakeholderanalyse
- Risikoanalyse
- Nutzwertanalyse
- Projektplan
- Weitere Nachweise (Screenshots, Testprotokolle usw.)